The `imgX` and `imgY` functions create an object for each image that stores an x value. This should remind you of D3 layout accessor functions. We'll build something more involved like this in chapter 9 and dive into writing layouts in chapter 10, but for this example we won't try to create an image gallery layout.

8.4.1 Interactively highlighting DOM elements

From here, we can add interactivity, such as making an image expand on mouseover. The process is rather simple.

Listing 8.11 Expand image on mouseover

```
function highlightImage(d) {
  var newWidth = parseFloat(d3.select("div.gallery")
                            .node().clientWidth);
  var imageSize = newWidth / 8;
  d3.select(this).transition().duration(500)
       .style("width", imageSize * 2)
       .style("background", "rgba(255,255,255,1)")
       .style("border-color", "rgba(0,0,0,1)");
  this.parentNode.appendChild(this)
};

function dehighlightImage(d) {
    var newWidth = parseFloat(d3.select("div.gallery")
                   .node().clientWidth);
    var imageSize = newWidth / 8;
    d3.select(this).transition().duration(500)
                .style("width", imageSize)
                .style("background", "rgba(255,255,255,0)")
                .style("border-color", "rgba(0,0,0,0)");
};

d3.selectAll("img")
       .on("mouseover", highlightImage)
       .on("mouseout", dehighlightImage);
```

We have to recalculate the width, because that value isn't accessible in this function.

Moves the image up the DOM to ensure it's drawn above the images around it

We don't move the image back in the DOM, because it can't overlap when it's reduced in size.

If you're a savvy web developer, you've probably spotted an artifact from working with SVG in the code above. It's the `appendChild` trick that we need to use to make SVG elements draw in front of each other. We're using relative and absolute positioned DOM elements, so we don't need this, because CSS has a z-index that allows elements to be drawn in front of each other. But I wanted to keep `appendChild` to remind you that working with traditional DOM elements has benefits that SVG elements don't.

Another reason to use the DOM rather than a z-index for positioning is to highlight the array position value in the accessor functions in D3. You may think that the array position corresponds to the array position of a datapoint in the original JavaScript array that we bound to the selection, but it doesn't. It corresponds to the array position of the DOM element in the selection. When you start to use `appendChild` to shift elements up and down the DOM, you change that array value. When we first created `imageArray`, we set the x value equal to the original array position, and